

Java OOPS(Object Oriented Programming Systems)

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects that contain data (fields) and behavior (methods). It focuses on designing software that closely represents real-world entities. It is used to:

- Improves code reusability
- Enhances maintainability and scalability
- Makes programs easier to understand and manage
- Closely models real-world entities

Characteristics of OOP

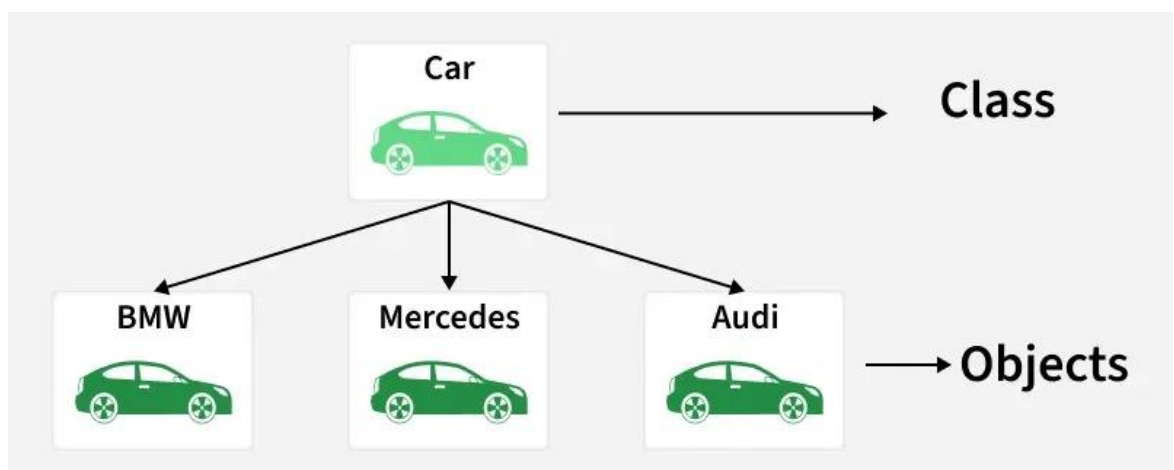
Object-Oriented Programming	Class	A blueprint or template for creating objects.
	Object	An instance of a class.
	Encapsulation	Protects data from direct access
	Abstraction	Hides unnecessary details
	Polymorphism	Allows the same method to behave differently
	Inheritance	Reuses code from existing classes

Class

A Class in Java is a blueprint or template used to create objects. It defines the properties (data) and behaviors (methods) that objects of that class will have.

- A class is a user-defined data type.
- It contains variables (fields) and methods.
- Multiple objects can be created from a single class.

Example: A Car represents a class (blueprint), while BMW, Mercedes, and Audi represent objects (instances) created from that class.

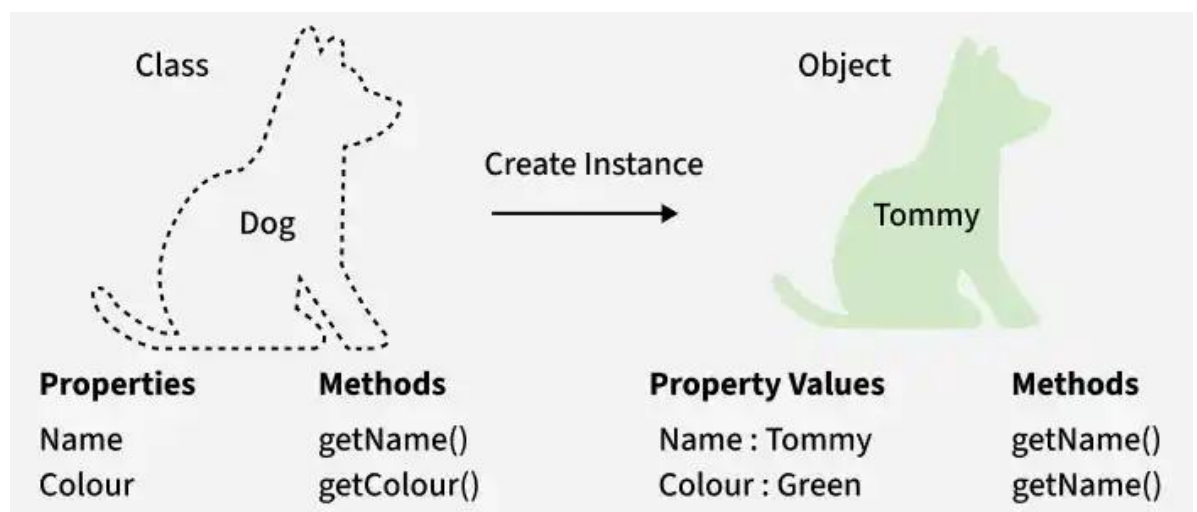


Object

An Object in Java is an instance of a class that represents a real-world entity. It is used to access the variables and methods defined inside a class.

- **State:** Represents the current data or attributes of an object.
- **Behavior:** Represents the actions that an object can perform.
- **Identity:** Every object has a unique identity in memory that distinguishes it from other objects, even if they contain the same data.

Example: Dog is a class, Tommy is an object of that class.



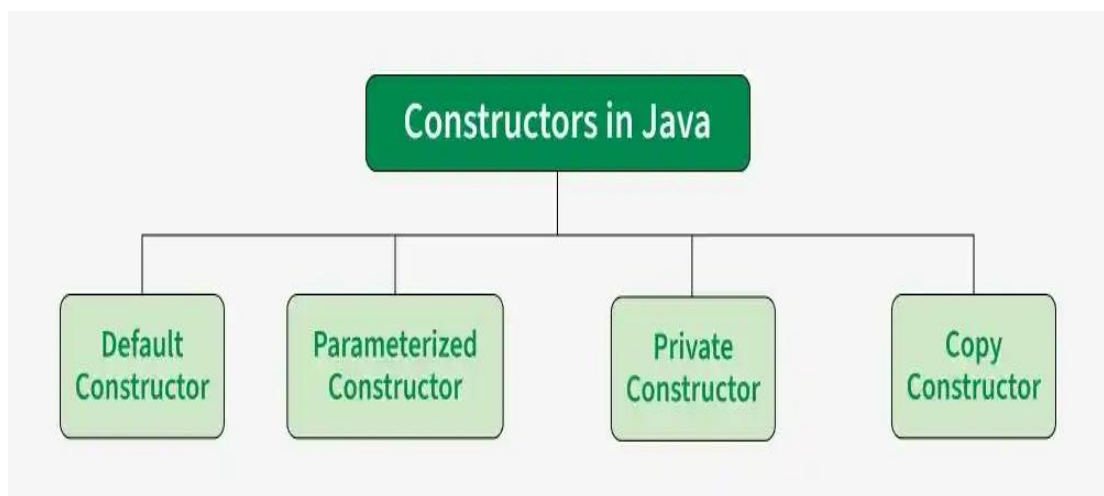
Constructor

A [Constructor](#) is a special member of a class that is automatically invoked when an object is created. It is primarily used to initialize the object's data members and set up the initial state of an object.

- Constructor name must be the same as the class name.
- It does not have any return type, not even void.
- It is automatically called when an object is created using the new keyword.
- Constructors help initialize object properties during object creation.

Example: When creating a Car object, a constructor can automatically assign values such as brand, model, and color.

Types of Constructor



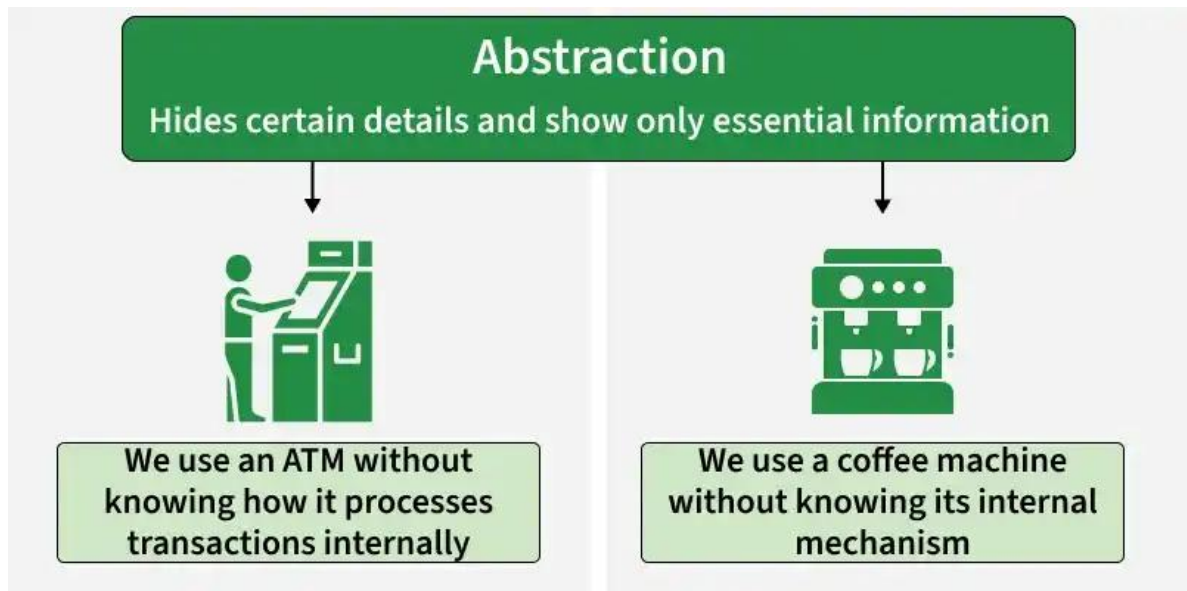
- **Default Constructor:** A constructor that does not accept any parameters and initializes an object with default values.
- **Parameterized Constructor:** A constructor that accepts parameters to initialize an object with specific values.
- **Copy Constructor:** A user-defined constructor that creates a new object by copying the data of another object of the same class.
- **Private Constructor:** A constructor declared with the private access modifier that prevents object creation from outside the class.

Abstraction

[Abstraction in Java](#) is the process of hiding implementation details and showing only the essential features of an object. It helps users focus on what an object does rather than how it does it.

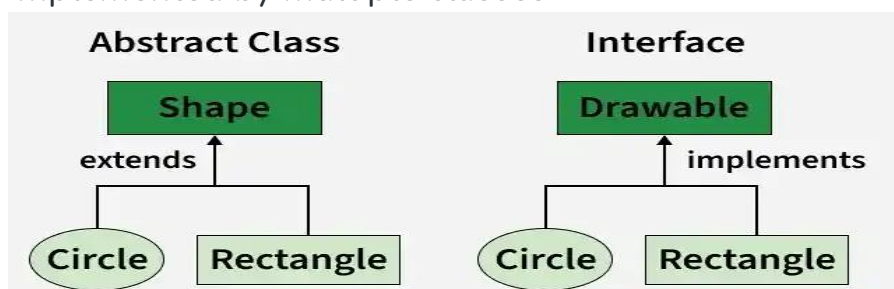
- **Hides complexity:** Internal implementation details are hidden from the user.
- **Improves maintainability:** Changes in implementation do not affect the user code.
- **Enhances flexibility:** Supports loose coupling through abstract classes and interfaces.

Example: An ATM or a coffee machine represents abstraction, where the user interacts with simple operations while the internal working and implementation details remain hidden.



How to Achieve Abstraction

- It is achieved in Java using [abstract classes](#) and [interfaces](#).
- [Interfaces](#) can provide 100% abstraction, while abstract classes provide partial abstraction.
- **Example:** Abstract class defines a common base (Shape) using inheritance, while an interface (Drawable) defines behavior implemented by multiple classes.



Encapsulation

Encapsulation is the process of wrapping data and methods into a single unit, usually a class, and restricting direct access to the data. It acts as a protective shield that prevents data from being accessed directly from outside the class.

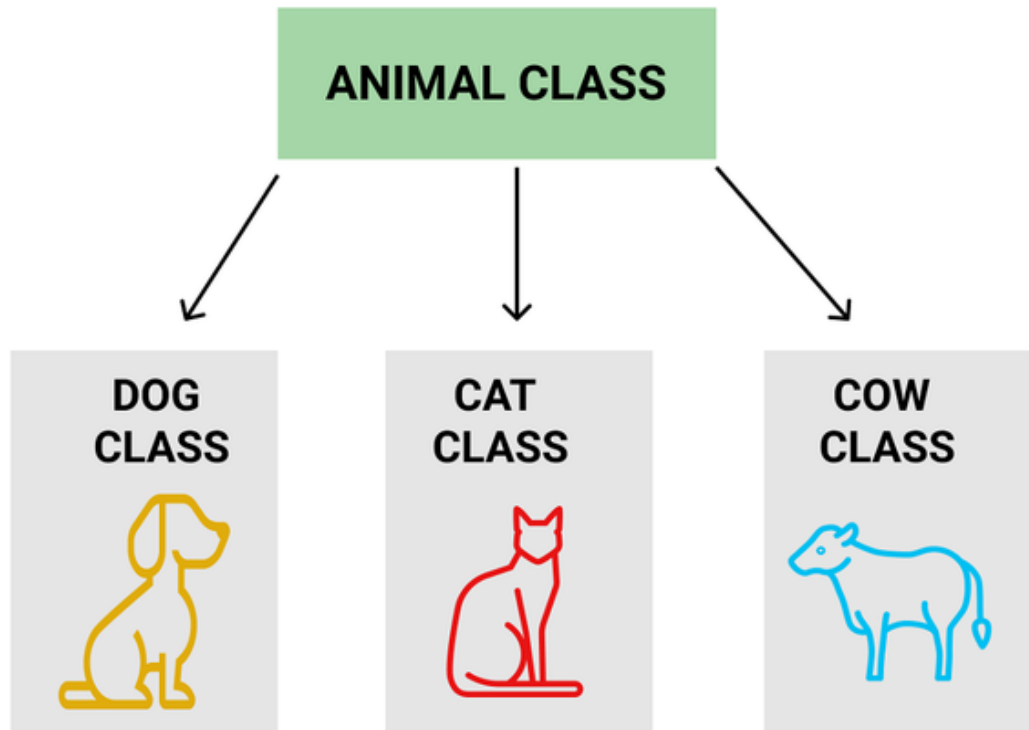
- Data members are hidden using the private access modifier.
- Access to data is provided through public getter and setter methods.
- It improves data security, maintainability, and controlled access.

Inheritance

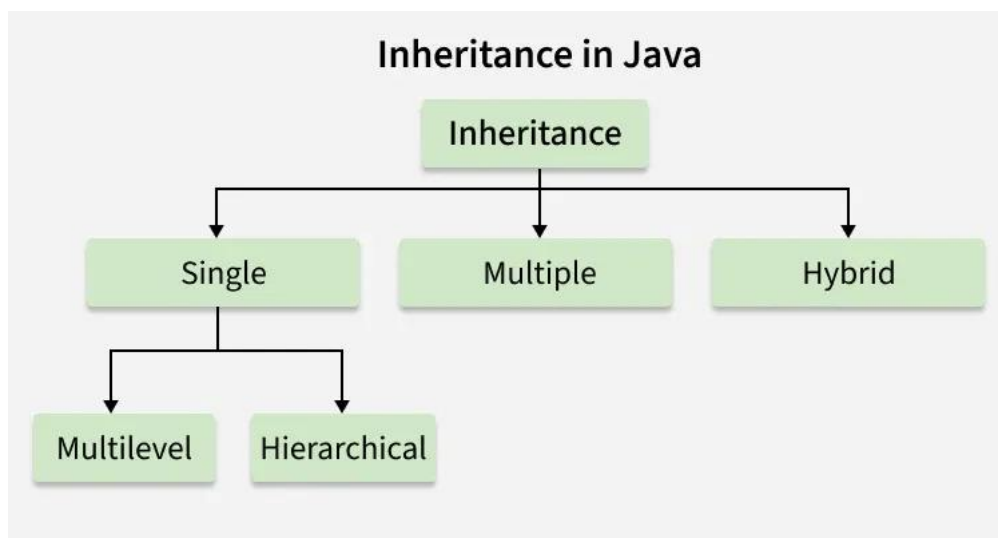
Inheritance is a core OOP concept in Java that allows one class to acquire the fields and methods of another class using the extends keyword. It represents an “is-a” relationship between classes.

- The class being inherited is called the superclass, and the inheriting class is the subclass.
- A subclass can use existing features of the superclass and also add its own.
- Inheritance promotes code reusability and reduces redundancy.

Example: Dog, Cat, Cow can be Derived Class of Animal Base Class.



Types of Inheritance



Java supports the following types of inheritance:

- **Single Inheritance:** One subclass inherits from one superclass.
- **Multilevel Inheritance:** A class is derived from another derived class, forming a chain.
- **Hierarchical Inheritance:** Multiple subclasses inherit from a single superclass.
- **Multiple Inheritance (through Interface):** A class inherits from multiple interfaces since Java does not support multiple inheritance using classes.
- **Hybrid Inheritance (through Interface):** A combination of two or more types of inheritance, achievable using interfaces.

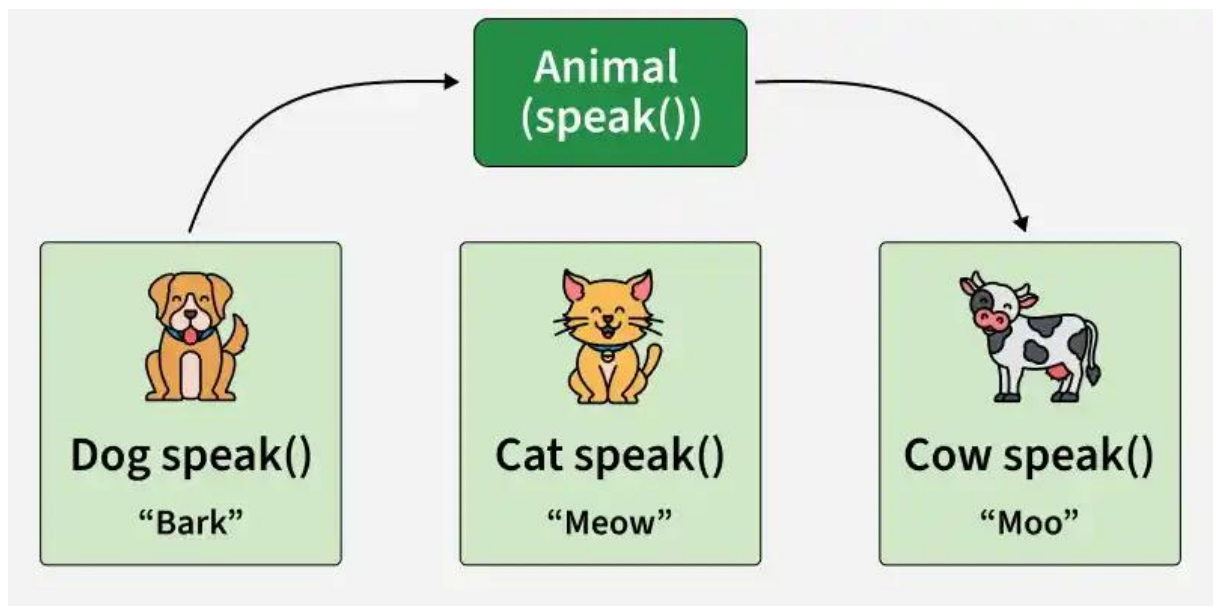
Polymorphism

Polymorphism means “many forms”, where a single entity can behave differently in different situations. In Java, it allows the same method or object to show different behavior based on context.

- Same method, different behavior depending on the object
- Achieved through method overloading and method overriding

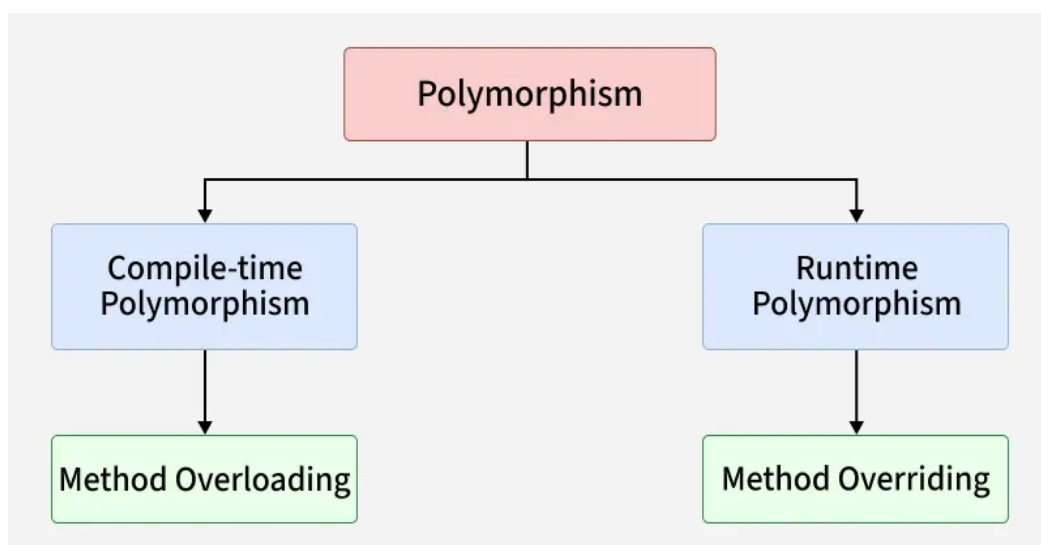
Example: Different animals represent polymorphism, where the same method `speak()` produces different

outputs like Bark, Meow, and Moo depending on the object.



Types of Polymorphism:

Polymorphism in Java is mainly of 2 types as mentioned below:



- **Compile-time Polymorphism**([Method Overloading](#))
:Achieved when multiple methods have the same name but different parameters. The method call is resolved at compile time.
- **Runtime Polymorphism** ([Method Overriding](#)): Achieved when a subclass provides a specific implementation of a method already defined in its superclass. The method call is resolved at runtime based on the object_

Advantage of OOP over Procedure-Oriented Programming Language

Object-oriented programming (OOP) offers several key advantages over procedural programming:

- **Code reusability:** Classes and objects allow reuse of existing code, reducing duplication and improving efficiency.
- **Better structure and maintainability:** Programs are organized into logical units, making code easier to understand, debug, and maintain.
- **Supports DRY principle:** Common functionality is written once and reused, leading to cleaner and more maintainable code.
- **Faster development:** Modular and reusable components help in quicker and scalable application development.

Limitations of OOP

While OOP provides many benefits, it also has some limitations:

- **Steeper Learning Curve:** Concepts such as classes, objects, inheritance, polymorphism, abstraction, and encapsulation can be challenging for beginners.
- **Additional Overhead for Small Programs:** OOP may introduce extra classes and structure that are unnecessary for simple applications.
- **Increased Design Complexity:** Designing a proper class hierarchy and object relationships requires careful planning.
- **Higher Memory Consumption:** Creating and managing a large number of objects can require more memory compared to procedural approaches.

